



Programming for Enginners

Spring 2026

Final Project Report

Mirana Rejepova, 61240017

15.05.2026

Introduction

“Typing of the Bombs” is a real-time typing game implemented in C. A plane flies across the top of the screen and drops bombs downward toward a city. Each bomb is paired with a randomly chosen codeword. The player must type the codeword letter-by-letter before the bomb reaches the city; correctly typed letters consume the word from the left (“book” > “ook” > “ok” > “k”), while mistyped letters carry a scoring penalty.

The project exercises several core systems-programming concepts: non-blocking keyboard input, frame-based screen rendering on a text console, file I/O for both text (codeword pool, ASCII art assets) and binary (save/load of game state), and modular C design across multiple translation units.

Implementation

1. Single-File Orchestration

The project is implemented within a single source file, `main.c`. This architecture was chosen to streamline the game state machine and simplify the compilation process for cross-platform environments. The code is logically structured into several specialized sections:

- **Constants & Structures:** definition of screen dimensions and the `GameState` structure.
- **Cross-Platform Compatibility:** support for Windows and macOS via `#ifdef _WIN32` macros.
- **Asset & Word Loaders:** handling external text file I/O for sprites and dictionaries.
- **Game Logic:** functions for object position updates and collision detection.
- **Rendering Engine:** frame generation via an in-memory character buffer.
- **Main Application Loop:** management of the menu and gameplay states.

2. Game State Representation

The entire dynamic state of the game is encapsulated in a single `GameState` structure. Crucially, this structure contains only primitive types and fixed-sized arrays, avoiding pointers. This pointer-free design allows the game state to be serialized directly to disk using `fwrite` and `fread`, enabling a robust binary save/load system that captures a perfect snapshot of the game.

3. Technical Systems

3.1 Non-Blocking Input & Timing

To ensure smooth, real-time animation, the game utilizes non-blocking input. Using custom implementations of `_khit()` and `_getch()` (adapted for both Windows and POSIX/macOS), the program polls for keystrokes without halting the main execution thread. Frame rates are regulated via `Sleep(60)`, maintaining a consistent update interval of approximately 60ms.

3.2 Double-Buffered Rendering

To eliminate the visual flicker typically associated with console-based games, a double-buffering technique is implemented. Instead of drawing directly to the console piecemeal, the `draw_screen` function populates a full-frame 2D character array (`screen[SCREEN_H][SCREEN_W]`). This complemented frame is then printed to the terminal in a single pass after a screen clear command, resulting in a stable image.

3.3 Scoring & Difficulty Progression

The scoring mechanism follows the requirement of calculating performance per word:

Score = Correct Letters – Mistakes

Difficulty scales dynamically through the `new_bomb` function. For every 5 bombs destroyed, the length of the next randomly selected codeword increases by 1 character, starting from a base length of 3 up to a maximum of 10.

4. Assets and File Handling

The program manages three distinct types of external data:

- **ASCII Art:** sprites for the plane, bomb, and city are loaded from .txt files. The loader strips `\r` and `\n` characters, ensuring compatibility across different operating systems.
- **Word Dictionary:** words are loaded from `codewords.txt`. The system validates each entry, filtering out non-alphabetic characters and words exceeding the buffer limit.
- **Binary Save System:** players can save their progress to any filename. The program writes the `GameState` structures as a binary blob for instantaneous reloading.

Discussion

Challenges Encourtered:

- **Cross-Platform Input:** the most significant challenge was implementing non-blocking input that works on both macOS (the development environment) and Windows (the target environment). This was solved by using preprocessor directives to swap between `conio.h` for Windows and a custom `termios.h` implementation for UNIX-based systems.
- **Screen Flickering:** initially, printing characters directly to the console caused heavy flickering. To resolve this, a frame-buffer approach was implemented, where the entire screen is prepared in a 2D array and printed in a single `printf` call.
- **Line Ending Compatibility:** text files created on different operating systems use different newline characters (`\r\n` vs `\n`). The asset loader was designed to strip these characters manually to prevent the ASCII art from breaking during rendering.

Potential Improvements:

- **Dynamic Dictionary:** currently, words are loaded once at the start. A future version could implement multiple difficulty levels with different dictionary files.
- **Color Support:** adding ANSI color codes to highlight correctly typed letters versus mistakes would improve the user interface.
- **High Score Table:** implementing a persistent “leaderboard” using a separate file to track the top scores.

AI Assistance Disclosure:

AI was used to consult on the implementation of the cross-platform input wrappers (`_khbit` and `_getch` for macOS) and to assist in structuring the logic for the double-buffered rendering system.

Compilation and Runtime Notes

This project was done using CLion:

1. Project Setup:

- Place `main.c`, `CMakeLists.txt`, and all `.txt` files (`plane.txt`, `bomb.txt`, `city.txt`, `codewords.txt`) in the root directory of the project.
- Ensure `CMakeLists.txt` contain the following:

```
File(COPY plane.txt bomb.txt city.txt codewords.txt DESTINATION ${CMAKE_BINARY_DIR}).
```

2. Execution:

- Select the “project_typingofthebomb” (`main.c`) target from the run configuration.
- Click Run (play button) or press “shift+f10”.

3. Save/Load files:

- When saving or loading a game, the files will be created in the “cmake-build-debug” (or “cmake-build-release”) folder unless a full path is specified.